

Java classes - Part 3

Page No :-

Date :- / /

- POJO class
- Enum class
 - Normal Enum class
 - Enum with custom values
 - Method override by constants
 - Abstract method in Enum
 - Interface implementation in Enum
 - What's the advantage of Enum class
- Final class
- Singleton class
 - Eager Initialization
 - Lazy Initialization
 - Synchronization Block
 - Double lock
 - Bill Pugh Solution
- Immutable class
- Wrapper class.

POJO Class :

means simple class

- Stands for "Plain Old Java Object".
- Contains variables and its getter and setter methods.
- class should be public
- Public default constructor
- No annotations should be used like @Table, @Entity, @Id etc.
- It should not extend any class or implement any interface.

Ex: public class Student {
 String name;
 private int roll;
 protected String address;

POJO class

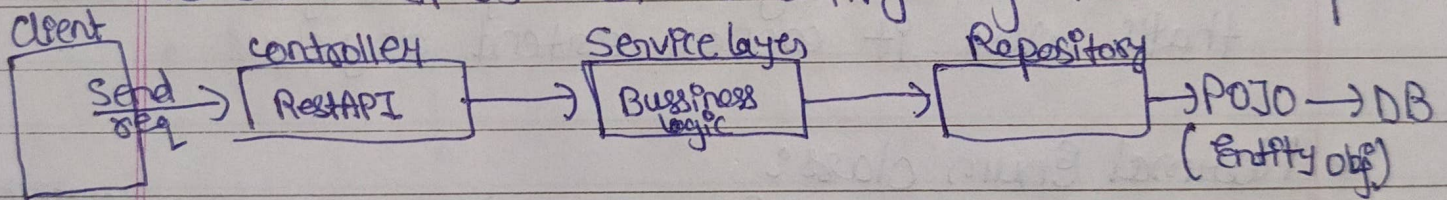
public ~~int~~^{String} getName() {
 return this.name;
 }

public void setName(String name) {
 this.name = name;
 }

Similarly getters & setters for roll & address

}

Usecase: It is used as Entity object in the Repository



- ② When a req comes from client then it is advisable to make use of POJO class (map req in POJO class) and use pojo for executing the req.

ENUM are 'Static final' by default
Constants are defined using static final

ENUM class :

- It has a collection of CONSTANTS (variable which values can not be changed)
- Its CONSTANTS are static and final implicitly
- It can not extend any class, as it internally extends java.lang.Enum class
- It can implement interfaces.
- It can have variables, constructors, methods.
- It can not be initiated (as its constructor will be private only, even you give default, in bytecode it make it private)
- No other class can extend Enum class
- It can have abstract method, and all the constant should implement that abstract method

A class cannot extend more than one class and Enum already extends java.lang.Enum class that's why it cannot extend any class

• Normal Enum class :

Ex - `public Enum Days {`

Default Value	Ordinal	name
0	0	MONDAY
1	1	Tuesday
2	2	Wednesday
3	3	Thursday
4	4	Friday
5	5	Saturday
6	6	Sunday

Ordinal

name

All in upper case

Here we can see there is no constructor
⇒ when this file compile and its .class file created when we open .class file compiler by default add private constructor internally.

`private Days() { }`

← semi colon at the end


```
public class Main {
    Psumc }

```

/* Common functions which is used

Days[] ← returns - values() = getting each enum in array form | Iterating on Enum
 - ordinal() = getting default value of enum (0, 1, 2, ...) ^{like}
 - valueOf() = used to match value by iterating on enum (late, consist, use)
 - name() = return what is the name of the variable
 */

// 1. usage of values() and ordinal()

```
for (Days day : Days.values()) {
    sout(day.ordinal());
}

```

// 2. usage of valueOf() and name()

```
Days days = Days.valueOf("FRIDAY");
sout(days.name());
}

```

```
}

```

output :

0

1

2

...

6

FRIDAY

ENUM with custom values:

```

Public enum EnumSample {
    MONDAY (101, "1st day of week"),
    TUESDAY (102, "2nd day of week"),
    WEDNESDAY (103, "3rd day of week"),
    THURSDAY (104, "4th day of week"),
    FRIDAY (105, "5th day of week"),
    SATURDAY (106, "6th day of week"), // ( ) is constructor
    SUNDAY (107, "7th day of week");
}

```

```

    private int val;
    private String comment;
    EnumSample (int val, String comment) {
        this.val = val;
        this.comment = comment;
    }

```

```

    public int getVal() {
        return this.val;
    }

```

```

    public String getComment() {
        return this.comment;
    }

```

```

    public static EnumSample print (int value) {
        for (EnumSample sample : EnumSample.values()) {
            if (sample.val == value) {
                return sample;
            }
        }
        return null;
    }
}

```



```

public class Main {
    Psum() {
        EnumSample sampleVar = EnumSample.print(107);
        sout(sampleVar.getComment());
    }
}

```

Output: 7th day of week.

Method Override by constant

```

Ex: public enum EnumSample {
    Monday {
        @Override
        public void dummyMethod() {
            sout("Monday dummy Method");
        }
    },
    TUESDAY,
    :
    SUNDAY;
    public void dummyMethod() {
        sout("default dummy method");
    }
}

```

```

Public class Main {
    psum() {
        EnumSample ob1 = EnumSample.FRIDAY;
        ob1.dummyMethod();
        EnumSample ob2 = EnumSample.MONDAY;
        ob2.dummyMethod();
    }
}

```

Output: default dummy Method
Monday dummy Method.

Enum with Abstract method:

- If Enum class has abstract method, then it is responsibility of all Enum values to provide the definition of abstract method.

Ex:

```

public enum EnumSample {
    MONDAY {
        public void dummyMethod() {
            sout("In Monday");
        }
    },
    SUNDAY {
        public void dummyMethod() {
            sout("In Sunday");
        }
    },
    public abstract void dummyMethod();
}

```

↳ normally child class resp to define definition of abstract method but ~~not~~ in enum. ↗

```

public class Main {
    public static void main() {
        EnumSample obj = EnumSample.MONDAY;
        obj.dummyMethod();
    }
}

```

Output: In Monday.

Here, we can see that every Enum variable has to provide defⁿ of abstract method, but this problem is solved by using Enum with Interface.

Enum Implement Interface:

```
ex: public interface MyInterface {
    public String toLowerCase();
}
```

```
public enum EnumSample implements MyInterface {
    MONDAY,
    TUESDAY,
    :
    SUNDAY;
}
```

@Override

```
public String toLowerCase() {
    return this.name().toLowerCase();
}
```

→ this is for
all Enum
constants

```
}
```

```
public class Main {
```

```
    psvm() {
```

```
        EnumSample mondayEnum = EnumSample.MONDAY;
```

```
        sout(mondayEnum.toLowerCase());
```

```
    }
```

```
}
```

Output: monday